

引理库相关工作综述

前提选择（Premise Selection）与引理库演化（Library Learning）

文档说明：本文件是为项目的”引理库的建立与管理”模块所做的文献调研，覆盖**前提选择**与**引理库演化**两条技术线。检索方式为 arXiv API 关键词检索（主题词包括 premise selection、retrieval + theorem proving、lemma library、submodular 等），并对关键论文下载全文精读；本次共下载并入藏 22 篇，清单见附录 A。本文件与《SG-Lean 思路汇报》配套阅读：后者讲我们打算怎么做，本文件讲别人已经做到了哪一步。

目录

1 综述范围与方法	2
1.1 我们要回答的问题	2
1.2 一个关键澄清：这是两条线，不是一条	2
1.3 一个被反复引用的痛点	2
2 前提选择（线 A）	3
2.1 奠基：把前提选择建成二分类问题	3
2.2 神经引导的符号证明器	3
2.3 对比学习与 Transformer	3
2.4 检索增强证明的范式确立	4
2.5 Lean 生态的检索工具爆发（2023 至 2026）	4
2.6 线 A 小结	5
3 引理库的演化与管理（线 B）	6
3.1 有用性驱动的引理挖掘	6
3.2 成长型技能库	6
3.3 wake-sleep：把库的演化做成两阶段循环（最接近我们的工作）	7
3.4 folklore 引理的生产线	8
3.5 把新验证的证明索引进检索记忆	8
3.6 分解式证明：库的另一种用法	8
3.7 领域专用的可复用证明库	9
3.8 线 B 小结：库的生命周期对照表	9

4 两线的交汇与横向对照	9
4.1 按”库的使用方式”分类	9
4.2 评测协议对照	9
5 从这条赛道得到的六条硬结论	9
6 我们的定位	11
6.1 已被占用的部分（不要声称）	11
6.2 仍然可能是空位的部分	11
6.3 必须正面处理的两条威胁	11
6.4 对当前项目方案的三条修改建议	12
A 本次检索并入藏的论文清单	12

1 综述范围与方法

1.1 我们要回答的问题

项目在”引理库”这条线上原本的设想可以拆成四个动作：（1）**建**：从一个课程语料出发，产出候选引理；（2）**选**：在库容有限的前提下决定留下哪些；（3）**管**：排序、去重、淘汰、版本；（4）**用**：在线解题时把库里的引理喂给证明器。

本综述要回答的是：这四个动作，学术上分别被谁做过了，做到什么程度，用的什么判据，报了什么数字。只有把这条基线摸清，才能判断我们剩下的可主张范围是什么。

1.2 一个关键澄清：这是两条线，不是一条

检索过程中很快会发现，”引理库”在文献里被拆成两个独立的研究方向，各自都有十年以上的积累：

技术线	回答的问题	典型输出
线 A：前提选择	给定一个证明状态或命题，从一个 大库里挑出对证明有用的前提	一个排序好的前提列表（每次查询返回前 k 条）
线 B：引理库演化	主动发现、抽象、压缩、淘汰引理， 形成一个小而精的库	一个可持续发展的引理库本身

两条线的交汇点在”用”：线 B 产出库，线 A 决定库里哪几条进上下文。我们的设想同时压在这两条线上，所以两条都要看。

1.3 一个被反复引用的痛点

这条赛道上有一个反复出现的经验结论，值得单独记住，因为它是我们项目的立论基础之一：

从证明过程中自动抽取出来的引理，大多问题特异、不可复用。LEGO-Prover 在 2023 年明确写出这一点，并以此作为设计动机；DreamProver 在 2026 年仍以“现有方法要么依赖固定库，要么合成的中间引理高度特异、缺乏通用性”作为出发点。

也就是说，“抽取的引理不可复用”是八年前就被看见、至今仍被当作开放问题的事。这既说明我们的问题意识是对的，也说明这条路上已经站着很多人。

2 前提选择（线 A）

2.1 奠基：把前提选择建成二分类问题

DeepMath, arXiv:1606.04442, 2016（Google, 含 Urban）

做法：把前提选择形式化为二分类：将待证命题与每个候选前提分别嵌入，用卷积、循环或混合网络编码，再拼接送入一层 logit 判断相关性。意义：这是“用神经网络做前提选择”的开创性工作，确立了此后十余年的基本范式：**嵌入 + 相关性判别**。对我们的意义：我们的“库”如果只是一个可检索的引理集合，那么它的检索方式必须与这套范式比较，而我们的起点是零（目前连检索都没有）。

FormulaNet 及图网络系列, 2017 至 2022

做法：形式化语句天然是树或图。此后一条支流把语句解析成树或图，用树结构网络或图神经网络编码：FormulaNet 用保留边序的图嵌入；Olšák 等人在一阶逻辑公式上构造能捕捉多种逻辑不变性的图网络框架；Paliwal 等人系统比较了 HOL Light 中公式的多种图表示；后续工作引入图对比学习；Ferreira 与 Freitas 以及 Bauer 等人则在大规模语料上构造**依赖图**，把定理与前提作为节点、依赖关系作为边，用图网络预测节点之间的连边来完成前提选择。对我们的意义：“依赖图”这个结构与我们想用的“引理被谁用到”的统计高度相关，需要明确我们与它的区别：他们用图结构做预测，我们想做的是用**实测覆盖增益**做筛选。

2.2 神经引导的符号证明器

The Isabelle ENIGMA, arXiv:2205.01981, 2022

做法：把学习方法与定理证明器紧密结合：为 Isabelle 的 Sledgehammer 问题训练**定制的 ENIGMA 引导、定制的神经前提选择**，并为 E 证明器设计针对性策略。训练数据是从 Isabelle 中抽取的数十万条无类型与有类型一阶问题，并**迭代多轮训练**。结果：最终最好的单策略 ENIGMA 与前提选择系统，在 15 秒内**比此前最好的 E 提升 25.3%**，同时超过所有已有的 ATP 与 SMT 系统。对我们的意义：这是“学习式前提选择确实能显著提升证明能力”的早期强证据，也说明这类增益必须在**统一时间预算**下比较才有意义（他们明确写了 15 秒）。

2.3 对比学习与 Transformer

Magnushammer, arXiv:2303.04488, 2023

做法：用 Transformer 与对比学习做前提检索，明确以“避免符号方法与工程开销”为卖点。结果：PISA 基准 **59.5%** 对 Sledgehammer 的 **38.3%**；miniF2F 基准 **34.0%** 对 **20.9%**；再与

语言模型证明器结合, 把 PISA 上的最好证明成功率从 57.0% 提到 **71.0%**, 而参数量少四倍。此外发布了当时最大规模的前提选择数据集 (证明状态与相关前提的文本对)。对我们的意义: ”检索做得好, 末端证明成功率就能被推上去” 在这里已有量化证据; 也说明检索器本身的训练数据规模是这类方法的关键资源。

2.4 检索增强证明的范式确立

LeanDojo 与 ReProver, arXiv:2306.15626, 2023

做法: 提出检索增强的 Lean 证明器 ReProver。关键设计是用程序分析识别可访问前提并构造困难负样本, 使检索器的训练更有效 (基于稠密段落检索 DPR)。同时构造了包含 98,734 条定理的基准。结果: 检索显著优于无检索版本。对我们的意义: ”可访问前提 + 困难负样本” 是训练检索器的标准配方; 如果我们要自己做检索, 这是必须复用的工程经验。

2.5 Lean 生态的检索工具爆发 (2023 至 2026)

这是与我们最直接相关的一节。近三年 Lean 侧的检索工具密集出现, 功能已经相当完整。

Machine-Learned Premise Selection for Lean, arXiv:2304.00994, 2023

做法: 在 Lean 内部实现在线训练的随机森林, 数据从 Mathlib 抽取, 以 `suggest_premises` tactic 的形式在编辑器里供用户调用。设计原则写得很清楚: 与证明助手紧耦合、易安装、轻量且快。对我们的意义: ”轻量、可嵌入、在证明助手内直接可用” 是一条已被验证的产品化路线。

LeanExplore, arXiv:2506.11085, 2025

做法: Lean 声明检索引擎, 混合排序: 多源语义嵌入 (形式代码、docstring、AI 生成的非形式化翻译、声明标题) 加 BM25+ 词法相关性加基于 PageRank 的重要度分数 (用声明的相互连接程度反映重要性)。覆盖 Batteries、Init、Lean、Mathlib、PhysLean、Std。提供网站、Python API 与 MCP, 可自托管。对我们的意义: 这一条直接对应我们设想中的 ”给常用的引理更高的地位”。他们用的是图上的 PageRank, 而不是实测使用频率。这是一个可以直接对照的判据差异。

Lean Finder, arXiv:2510.15940, 2025

做法: 面向数学家意图的语义搜索: 先分析并聚类公开 Lean 讨论, 再用合成查询微调文本嵌入, 并用多种反馈信号对齐用户偏好。结果: 在真实查询、非形式化陈述与证明状态三类评测上, 相对此前检索引擎与 GPT-4o 相对提升超过 30%。对我们的意义: 检索质量的下一个瓶颈是 ”查询与库表示之间的意图错配”, 这条对我们设计提示词与查询构造有直接参考价值。

LeanPremise 与 LeanHammer, arXiv:2506.07477, 2025

做法: LeanPremise 是首个面向 hammer 使用场景、针对依赖类型论训练的前提选择器。关键特性是动态适应用户上下文: 运行时从用户环境动态抽取签名, 从而推荐训练数据之外的库以及用户本地定义的引理。结果: 相比已有前提选择器, 使 LeanHammer 多解 21% 的目标; 并在 miniCTX-v2 上评测了泛化能力。对我们的意义: ”动态适应用户上下文” 几乎就是我们要做的事: 我们的库本质上就是 ”训练数据之外的、动态积累的引理集合”。这一点必须正面处理。

LeanSearch v2, arXiv:2605.13137, 2026

问题定义: 论文明确指出既有工作只覆盖了两个相邻问题: 语义检索引擎找的是单条匹配声明的声明; 前提选择系统预测的是单步 tactic 需要的前提。两者都不能给出 ”一整条定理所需的前

提集合”。作者把这件事命名为**全局前提检索**。做法：两模式。标准模式：构建**层级化非形式化**的 Mathlib 语料，用嵌入加重排的两阶段流水线，且**不做领域微调**。推理模式：以标准模式为检索底座，做迭代的**草图、检索、反思**循环，为目标定理生成证明草图，再为草图的每一步检索候选、过滤并评判，迭代修订草图直到得到一致的支撑引理集合。结果：标准模式 nDCG@10 达 **0.62**（次优系统 0.53）。在 69 条研究级 Mathlib 定理查询的基准上，推理模式在 10 个候选内恢复 **46.1%** 的真实前提组（对比：推理式检索器 38.0%，前提选择基线 9.3%）。下游验证（最重要的一条）：在固定的证明循环里替换检索器：LeanSearch v2 取得最高证明成功率 **20%**，次优系统 16%，无检索只有 **4%**。对我们的意义：这是全篇最值得我们记住的数字。它同时说明两件事：（1）**检索质量确实直接传导为证明成功率**，杠杆很大；（2）“按需检索出相关引理集合”这件事已经被做到相当深，不能作为我们的创新点。

Combining Textual and Structural Information, arXiv:2510.23637, 2025

做法：图增强的前提选择：用稠密文本嵌入结合**异构图**上的图神经网络，图中同时包含”状态与前提”和”前提与前提”两类关系。结果：在 LeanDojo 基准上，相对 ReProver 的语言基线，标准检索指标**提升超过 25%**。对我们的意义：再次说明”N 条引理之间的关系（依赖、共现）”是有效信号，而这恰好也是”库的结构化组织”能提供的东西。

Semantic Search over 9 Million Mathematical Theorems, arXiv:2602.05216, 2026

做法：在规模上做文章：把 arXiv 等八个来源的 **920 万**条定理陈述统一成语料，用”简短自然语言描述”作为检索表示，系统分析表示上下文、语言模型选择、嵌入模型与提示策略的影响。结果：在职业数学家撰写的查询集上，定理级与论文级检索都大幅优于已有基线。对我们的意义：说明”为每条陈述生成一句自然语言描述再检索”是一个稳定有效的表示，我们若要给库中的引理建索引，这是可直接借鉴的做法。

CSLibPremiseBench, arXiv:2605.14549, 2026（审计型负面结果）

做法：针对 Lean 4 的计算机科学库 CSLib 构建可复现的前提检索基准：固定 CSLib v4.29.0，任务集含 801 条可标注的定理与引理任务、1875 条候选声明。对比 BM25、符号与命名重叠、命名空间与模块启发式、PageRank 式模块先验、固定混合方案，以及作者自提出的结构引导图词法重排器 CSG-Rerank。标签可靠性审计：他们做的这件事很值得学：标签是”源码可见的证明引用代理”而非 elaborate 后的依赖轨迹；严格审计保留 2666/2845（93.7%）的代理链接，但用 300 条任务做 Lean 环境表达式探针时，只在 958 条代理链接中观察到 462 条（48.2%）。结果：结构引导的重排只在严格策略下带来微弱的早期排名提升，**无法可靠超过 BM25 加符号重叠**，Recall@10 也没有改善。对我们的意义：**两条警示**：一是朴素的结构化启发式（命名空间、模块、图先验）**不可靠**，别指望”领域划分 + 命名规则”就能提升检索；二是**标签本身可能非常不可靠**，任何关于”检索命中率”的指标都必须先审计标签。

2.6 线 A 小结

结论：线 A 已经成熟到”有公开检索引擎、有 API、有 MCP、有基准、有负面结果”的程度。凡是”给引理排序、按需检索、动态适配、重要度加权”这类想法，**都被占**。

阶段	代表	方法要点
二分类奠基	DeepMath	嵌入 + 相关性判别
结构化表示	FormulaNet、GNN 系列、依赖图	树/图编码，图对比学习，连边预测
神经引导符号器	Isabelle ENIGMA	学习引导 + 前提选择 + 定制策略，统一时限比较
对比学习	Magnushammer、PACT	Transformer + 对比/自回归目标
检索增强证明	LeanDojo / ReProver	可访问前提 + 困难负样本
Lean 生态工具	LeanExplore、Lean Finder	混合排序（语义 + BM25+ + PageRank）、意图对齐
端到端 hammer	LeanPremise / LeanHammer	为依赖类型论训练，动态适应用户上下文
全局前提检索	LeanSearch v2	一次给出一组前提，迭代草图与检索，下游增益显著
基准与审计	CSLibPremiseBench	负面结果：结构启发式不可靠；标签需审计

3 引理库的演化与管理（线 B）

这一节与我们的模块最直接相关，按时间顺序展开。

3.1 有用性驱动的引理挖掘

Learning-assisted Theorem Proving with Millions of Lemmas, arXiv:1402.3578, 2014

做法：大型形式化库里存在数百万条已证语句，但与人写数学一样，其中只有极小一部分被命名并在后续证明中被复用。作者**提出并实现了一套判据**，用来估计 HOL Light 引理对证明后续定理的“有用性”，并据此在 HOL Light 与 Flyspeck 的**大规模推理图**中挖掘出最好的引理，加入可复用的语句池，再配合基于学习的相关性过滤。结果：这种组合显著增强了在 Flyspeck 这类大型形式化库上对**新猜想**的自动证明（作为背景：当时 HOLyHammer 能证明 Flyspeck 中约 47% 的定理）。对我们的意义：**这是全篇最需要警惕的一条**：“按有用性给引理打分并筛选入库”是十二年前就有的想法。我们如果只是把“有用性”换成另一个名字，没有任何新意。

3.2 成长型技能库

LEGO-Prover, arXiv:2310.00656, 2023

做法：指出现有方法的共同局限是**假设定理库在整个证明过程中固定不变**。LEGO-Prover 维护一个**不断增长的技能库**（已验证的引理作为技能）：模块化地构造证明，使模型既能检索使用已有技能，又能在证明过程中**创建新技能**；这些技能还会被进一步“演化”（通过提示语言模型）以变得更通用、更可复用，从而在另一个尺度上丰富库。此外，学到的库还能弥合人类证明与形式证明之间的差距，使缺失步骤更容易被填补。结果：miniF2F 通过率：valid 从 48.0% 提到 **57.0%**，test 从 45.5% 提到 **50.0%**；过程中生成**两万多个技能**并加入库；消融显示新加入的技能确实有用：成功率从 47.1% 提升到 **50.4%**。局限（论文自述，也是我们的机会）：从证明器抽取出的引理“大多是问题特异的，因而不可复用”，所以它们要额外加一个“演化器”来提升通用性。对我们的意义：**这是“库的建立 + 库的使用”的第一篇完整工程实现**，而且它给出了一个**可直接对照的增益量级**：库的净贡献约 3 个百分点。

3.3 wake-sleep: 把库的演化做成两阶段循环（最接近我们的工作）

DreamProver, arXiv:2604.26311, 2026

这是本次调研中对我们的设想覆盖最完整的一篇，因此单独展开。

问题陈述：现有方法要么依赖**固定**的引理库（缺乏适应性），要么合成的中间引理**高度特异**（缺乏通用性）。

总体框架：用“wake-sleep”程序归纳范式，构成迭代的两阶段循环：

- **wake 阶段**：系统尝试用**当前库**去证明训练集中的定理，同时提出新的引理候选（来自证明过程中的中间定理）。
- **sleep 阶段**：系统从这些经验中学习，把候选**抽象、精炼、整合**，以**压缩并优化库**。

sleep 阶段的第一步：引理抽象：采用基于聚类的演化策略：先给每条中间定理标注自然语言描述（含子领域、难度，例如“是否只是平凡的等价变换”或“是否需要更高级的推理”，以及可用于其证明的相关定理）；再对描述做嵌入并用余弦距离做 **K-Means 聚类**（簇数用肘部法确定）；然后对每个簇，把簇内定理及其描述作为上下文交给语言模型，生成**更一般、更可复用的**候选引理；最后用**结构化相似度**校验候选是否真的适用于该簇：把候选引理与簇内定理都转成简化的一阶逻辑表达式树，比较树之间的可对齐程度，最高相似度超过阈值才保留。

sleep 阶段的第二步：库更新（三步）：

1. **遗忘**：追踪每条引理在过去各个 wake 阶段的使用频率，采用**最近最少使用（LRU）**策略，当库达到容量上限时淘汰最少被使用的引理。论文明确说明这样做的理由之一是**避免把整个库作为输入上下文时把重要引理挤掉**。
2. **存入非重复项**：用**树编辑距离**衡量候选与库中已有引理的相似度，距离很小（即高度相似）的视为重复并移除，以降低冗余、保持多样性。
3. **形式化验证**：对剩下的每条候选引理，用与 wake 阶段相同的直接提示策略尝试证明它。

推理阶段：先用库做直接提示证明；若失败，退化为**草图与证明**工作流：先生成证明草图，再逐个证明子目标，两个环节都继续使用库。

关键规模数字：库被维持在**不到 100 条**引理。作者称这个遗忘机制“维持了一个简洁的引理库”，从而支持稳定高效的复用。

结果：相对此前的状态最优系统 Hilbert，在不平等、数论、组合三个领域分别提升 20%、114%、50%，**平均提升 61%**；相比 Goedel-Prover-V2，输出 token 分别减少 55%、56%、76%；相比 Hilbert 减少 42%、50%、53%；证明长度相比两者平均减少 32% 与 50%。数据设置：库是**按领域分别构建**的：每个领域从对应的训练子集采 **100 道题**；评测用 567NEQ、ChenNEQ、MO-INT（不等式）、PutnamBench 与 ProverBench 的数论子集、CombiBench 子集（组合）。泛化范围：论文用单独的研究问题考察“低资源领域的泛化”，但表述始终是“**相关领域的未见定理**”，并未声称跨远域泛化。

对我们的三点直接含义：

1. 我们设想的“库的建立、演化、淘汰、去重、验证”这条链，**已经被完整实现过一遍**，而且每一环都有具体判据：聚类与抽象用于选择，LRU 使用频率用于排序与淘汰，树编辑距离

用于去重，形式化验证用于准入。

2. 他们的选择判据是启发式：语义聚类与结构相似度阈值，不是实测的工作负载覆盖增益，也没有近似保证。差异点正在这里。
3. 他们被迫把库压到 100 条以内，原因是**整库要作为上下文输入**。他们选择压缩库来适配上下文；我们设想的是用检索从库里取前若干条。这是两种不同的解法，而后者正是线 A 已经证明有效的那一条。

3.4 folklore 引理的生产线

MathlibLemma, arXiv:2602.02561, 2026

做法：面向一个具体而实用的问题：Mathlib 里缺少大量**民间引理**（数学家习以为常、但形式库中不一定有的中间事实），这限制了 Lean 作为日常工具的可用性。作者提出模块化流水线，自动完成这类引理的发现、形式化与证明。结果：产出一个经验证的引理库，包含 **1506** 条通过”证明旁路筛查”（proof-bypass screen）的 Lean 检查通过的证明；其中一个小的精选子集已被合并进 **Mathlib**，作为”输出能达到专家库标准”的外部证据。此外构造了 MathlibLemma 基准：**4028** 条非平凡、通过类型检查的语句，覆盖广泛数学领域；对未被证明的残差做种子分层抽样审计，**78%** 在数学上成立（说明这些未解实例大多是有效命题而非幻觉）。在该基准上，所有模型合计证明 **37%**，而**单个模型最多约 19%**。对我们的意义：这是”自造引理入库”的当代最强形态，而且它证明了这类产出 **能达到被人工库接纳的质量**。同时它给出了一个有用的基准与一个可对照的口径：单模型上限约 19%，集合约 37%，说明这类任务仍有很大空间。

3.5 把新验证的证明索引进检索记忆

OProver, arXiv:2605.17283, 2026

做法：指出智能体式证明很少被纳入训练，通常只在推理时使用。OProver 把二者统一：失败的证明尝试用”检索到的、经编译器验证的证明”与 Lean 编译器反馈**迭代修正**；每一轮迭代都执行智能体式证明，把**新验证的证明索引进 proof 集合与检索记忆**，把修复轨迹用作监督微调数据，把未解的难题用于强化学习。规模与结果：其 OProofs 集合含 177 万条 Lean 语句、686 万条编译器验证的证明，以及带有检索上下文、失败尝试、反馈与修复过程的序列化轨迹。在五个基准上取得最好或次好的成绩，其中 MiniF2F 为 93.3%（Pass@32）。对我们的意义：它把”新证明入库”和”检索记忆”做成了训练闭环的一部分，说明库与检索记忆的持续增长是当前主流系统的标准组件。

3.6 分解式证明：库的另一种用法

Hilbert, arXiv:2509.22819, 2025

做法：编排四个组件：擅长度推理的非形式化语言模型、为 Lean 4 优化的专用证明模型、形式验证器、**语义定理检索器**。当证明模型无法解决时，用**递归分解**把问题拆成子目标，交给证明模型或推理模型解决，并利用验证器反馈精修错误证明。结果：miniF2F 达 **99.2%**（比当时公开最好高 6.6 个百分点）；PutnamBench 解出 462/660（**70.0%**）。对我们的意义：**这个数字很重要**：

miniF2F 已经被做到 99.2%，意味着该基准几乎没有增益空间。我们原定的”在 miniF2F 上测增益”的方案必须修改。

3.7 领域专用的可复用证明库

CircuitProver, arXiv:2607.27259, 2026

做法：硬件形式验证场景：把参数化硬件设计与自然语言规格转成可执行的 Lean 4 模型，再通过 Lean 反馈迭代构造机器检查的证明；把证明轨迹与已验证定理蒸馏成可复用的库，库中的证明策略指导后续智能体的推理，已验证引理可被复用；支持证明积累与参数化验证。对我们的意义：这是”库”在垂直领域里的应用形态，说明按领域建库在实践中是成立的（与 DreamProver 的按领域建库一致）。

3.8 线 B 小结：库的生命周期对照表

库生命周期的环节	谁做过	用的判据
建：产出候选引理	LEGO-Prover、Dream-Prover、MathlibLemma、OProver	证明过程中的中间定理；针对民间引理的定向挖掘；智能体式证明
选：决定留下哪些	DreamProver	语义聚类 + 抽象 + 结构化相似度阈值（启发式）
准入：能不能进	DreamProver、MathlibLemma	形式化验证 + 证明旁路筛查
去重	LEGO-Prover、Dream-Prover	字符串相似度；树编辑距离
排序与淘汰	DreamProver	LRU （跨轮次使用频率）
容量	DreamProver	硬上限，约 100 条，受提示词上下文约束
用：进上下文	DreamProver	整库注入 （所以必须压小）
用：按需检索	Lean 生态全体（见第三节）	语义嵌入 + 词法 + 图重要度等混合排序

两个空白可以直接看出来：（1）没有任何工作作用”实测的工作负载覆盖增益”来做选与汰，全是启发式；（2）库的容量上限是被”整库注入”这个用法逼出来的，如果改用检索，容量约束的性质就完全不同（从”能塞进上下文”变成”检索精度”）。

4 两线的交汇与横向对照

4.1 按”库的使用方式”分类

4.2 评测协议对照

5 从这条赛道得到的六条硬结论

1. 检索是已被验证的大杠杆，而我们还没有。LeanSearch v2 的下游对照给出了 20% 对 4%（无检索）的差距。我们的流水线目前连检索层都没有，这是最应该先补的一环。但正因为它成熟，

用法	代表与代价	
整库注入提示词	DreamProver。代价：库必须保持很小（他们在 100 条以内）	
逐步检索（tactic 级）	ReProver、LeanPremise、Magnushammer。代价：需要训练检索器	
全局检索（一次给一组）	LeanSearch v2 的推理模式。代价：迭代多次查询，调用成本上升	
把新证明索引进记忆	OProver。代价：需要持续维护索引与训练闭环	

工作	基准	关键数字
Magnushammer	PISA、miniF2F	59.5% 对 38.3%；34.0% 对 20.9%
Isabelle ENIGMA	Sledgehammer 问题集	15 秒内相对提升 25.3%
LEGO-Prover	miniF2F	valid 48.0 到 57.0；test 45.5 到 50.0；消融 +3.3 点
DreamProver	不等式、数论、组合	相对 Hilbert 平均 +61%；token 减少最多 76%
MathlibLemma	自建 4028 条基准	全模型合计 37%，单模型最多约 19%
Hilbert	miniF2F、PutnamBench	99.2%；70.0%
OProver	五个基准	MiniF2F 93.3%、PutnamBench 11.3%
LeanSearch v2	69 条研究级查询	nDCG@10 0.62；下游证明 20% 对 16% 对无检索 4%
CSLibPremiseBench	CSLib	结构重排无法可靠超过 BM25 加符号重叠

不能作为创新点。

2. ”库的建、选、管、用”每一环都有代表作。DreamProver 是最完整的一次实现，覆盖了聚类抽象、LRU 淘汰、树编辑距离去重、形式化验证准入与整库注入。我们原设想的清单基本被逐项覆盖。
3. 选择标准全部是启发式。使用频率（LRU）、语义聚类、结构相似度阈值、字符串相似度。没有工作用”实测的工作负载覆盖增益”并给出近似保证。这可能是一条真实的空位。
4. 整库注入是容量瓶颈的根源。DreamProver 明确因为”整库要进上下文”而把库限制在 100 条以内。这意味着”库能长多大”被提示词预算而非可用引理数量决定。改用检索替代压缩，是一个结构性差异。
5. **miniF2F 已饱和，不能作为我们的主基准。** Hilbert 99.2%、OProver 93.3%，天花板效应会淹没任何真实增益。需要改用有提升空间的基准（PutnamBench、CombiBench、ProofNet、ProverBench、FATE-M、567NEQ 等）。
6. 负面结果与标签审计必须纳入设计。CSLibPremiseBench 说明结构启发式（命名空间、模块、图先验）不可靠，且”源码可见的引用代理”标签与真实依赖只有约一半吻合。我们若报”库命中率”或”引用率”，必须先做标签可靠性审计。

6 我们的定位

6.1 已被占用的部分（不要声称）

想法	已被谁做
从大库里按相关度检索前提	ReProver、Magnushammer、LeanPremise、LeanExplore、LeanSearch v2
给引理按重要度排序	LeanExplore 用 PageRank; DreamProver 用使用频率 LRU
按需检索而非整库注入	Lean 生态的检索工具普遍如此
自造引理并入库	LEGO-Prover、DreamProver、MathlibLemma
库的去重	LEGO-Prover 用字符串相似度; DreamProver 用树编辑距离
库的淘汰	DreamProver 的 LRU 遗忘
库的压缩与抽象	DreamProver 的 sleep 阶段（比我们设想的更深）
把新验证的证明索引进可用记忆	OProver
分解式证明与库配合	Hilbert、DreamProver 的草图与证明流程

6.2 仍然可能是空位的部分

1. 用实测的工作负载覆盖增益来做库的选择与淘汰。现有工作的判据都是启发式（使用频率、结构相似度、语义聚类）。而”覆盖”是并集形式的函数，天然单调子模，因此贪心可给出 $1 - 1/e$ 的近似保证。把”有近似保证的优化”与”启发式排序”放在同一工作负载上对打，是一个干净的、可证伪的对比实验。本次 arXiv 检索（主题词含 submodular 与 data selection）未发现相关工作，但需注意 arXiv 检索只覆盖元数据，不能视为穷尽。
2. 需求驱动：从证明轨迹中挖掘跨目标反复出现的子目标，用它决定”造什么引理”。SGS 只是条件化在未解目标上，并不做需求挖掘与跨目标过滤；线 B 的工作基本是”从已产生的中间定理里事后筛选”。这个”事前定方向”的角度未见对应工作。
3. 用检索替代容量压缩。DreamProver 因整库注入而必须把库限制在 100 条以内；若库改为按需检索，容量约束就从”能否装进上下文”变为”检索精度”，库可以长得更大。这是一个尚未被检验的设计选择。
4. 形式化判据作为准入与排序。MathlibLemma 有”证明旁路筛查”，DreamProver 有结构相似度阈值，但没有人用一套”硬门乘软分”式的可计算判据同时承担准入、排序与淘汰。

6.3 必须正面处理的两条威胁

1. LeanPremise 的”动态适应用户上下文”与我们的库在功能上高度重叠。它是”从用户环境动态抽取签名以支持训练外库与本地引理”，我们做的是”积累自己的引理并喂给证明器”。定位时必须给出清晰区别，否则容易被判定为重复。

2. ”按有用性筛引理”有 2014 年的先例（HOL Light 与 Flyspeck 的百万级推理图）。我们若只把”有用性”换一个名字，不构成新意；差异必须在”面向工作负载的覆盖最大化 + 近似保证”这个层面。

6.4 对当前项目方案的三条修改建议

1. 检索层的优先级提到最高，而且从一开始就采用已公开的检索工具（如 LeanExplore 这类可自托管或带 API 的引擎），不要自己从零训检索器。理由：这条路已经成熟，自研的期望收益低。
2. 主基准从 miniF2F 换成有余量的基准，并在报告里按领域分桶。理由：miniF2F 已被做到 99.2%。
3. 把”库的选择层”当作唯一的主攻方向，用”实测覆盖 + 子模保证”与”启发式排序”做正面对照；库的其余环节（去重、淘汰、准入）直接沿用已有方案，不重复造。

A 本次检索并入藏的论文清单

以下 22 篇已下载到文献库目录 ATP_Papers/08_Library_Premise_Selection 下，编号接续原有清单（101 起）。

注：另有若干重要工作位于本地文献库的既有分类中，本次一并引用，包括：

- DeepMath, arXiv:1606.04442
- Isabelle ENIGMA, arXiv:2205.01981
- Magnushammer, arXiv:2303.04488
- LeanDojo 与 ReProver, arXiv:2306.15626
- LEGO-Prover, arXiv:2310.00656
- MINIMO, arXiv:2407.00695
- SGS, arXiv:2604.20209

编号	arXiv	标题
101	2604.26311	DreamProver: Evolving Transferable Lemma Libraries via a Wake-Sleep Theorem-Proving Agent
102	2605.13137	LeanSearch v2: Global Premise Retrieval for Lean 4 Theorem Proving
103	2506.07477	Premise Selection for a Lean Hammer
104	2506.11085	LeanExplore: A search engine for Lean 4 declarations
105	2510.15940	Lean Finder: Semantic Search for Mathlib That Understands User Intents
106	2605.14549	CSLibPremiseBench: Structure-Guided Premise Retrieval and Label Robustness for Lean 4
107	2510.23637	Combining Textual and Structural Information for Premise Selection in Lean
108	2602.05216	Semantic Search over 9 Million Mathematical Theorems
109	2602.02561	MathlibLemma: Folklore Lemma Generation and Benchmark for Formal Mathematics
110	2605.17283	OProver: A Unified Framework for Agentic Formal Theorem Proving
111	2509.22819	Hilbert: Recursively Building Formal Proofs with Informal Reasoning
112	2304.00994	Machine-Learned Premise Selection for Lean
113	2303.15642	Graph Sequence Learning for Premise Selection
114	2501.15797	LemmaHead: RAG Assisted Proof Generation Using Large Language Models
115	2502.17925	LeanProgress: Guiding Search for Neural Theorem Proving via Proof Progress Prediction
116	2310.04353	An In-Context Learning Agent for Formal Theorem-Proving
117	2604.19558	On Reasoning-Centric LLM-based Automated Theorem Proving
118	2601.14027	Numina-Lean-Agent: An Open and General Agentic Reasoning System for Formal Mathematics
119	1402.3578	Learning-assisted Theorem Proving with Millions of Lemmas
120	2505.20613	REAL-Prover: Retrieval Augmented Lean Prover for Mathematical Reasoning
121	2607.27259	CircuitProver: Agentic Lean 4 Theorem Proving with Reusable Circuit Proof Library
122	2510.10815	DRIFT: Decompose, Retrieve, Illustrate, then Formalize Theorems